



Web Programming 2 (C#)

Week 3

Web Programming 2 program

01 (wk-16) ViewModels / Passing data to a View

02 (wk-17) State Management

03 (wk-18) *break*

04 (wk-19) Services

05 (wk-20) Partial views / View Components

06 (wk-21) ...

07 (wk-22) ...

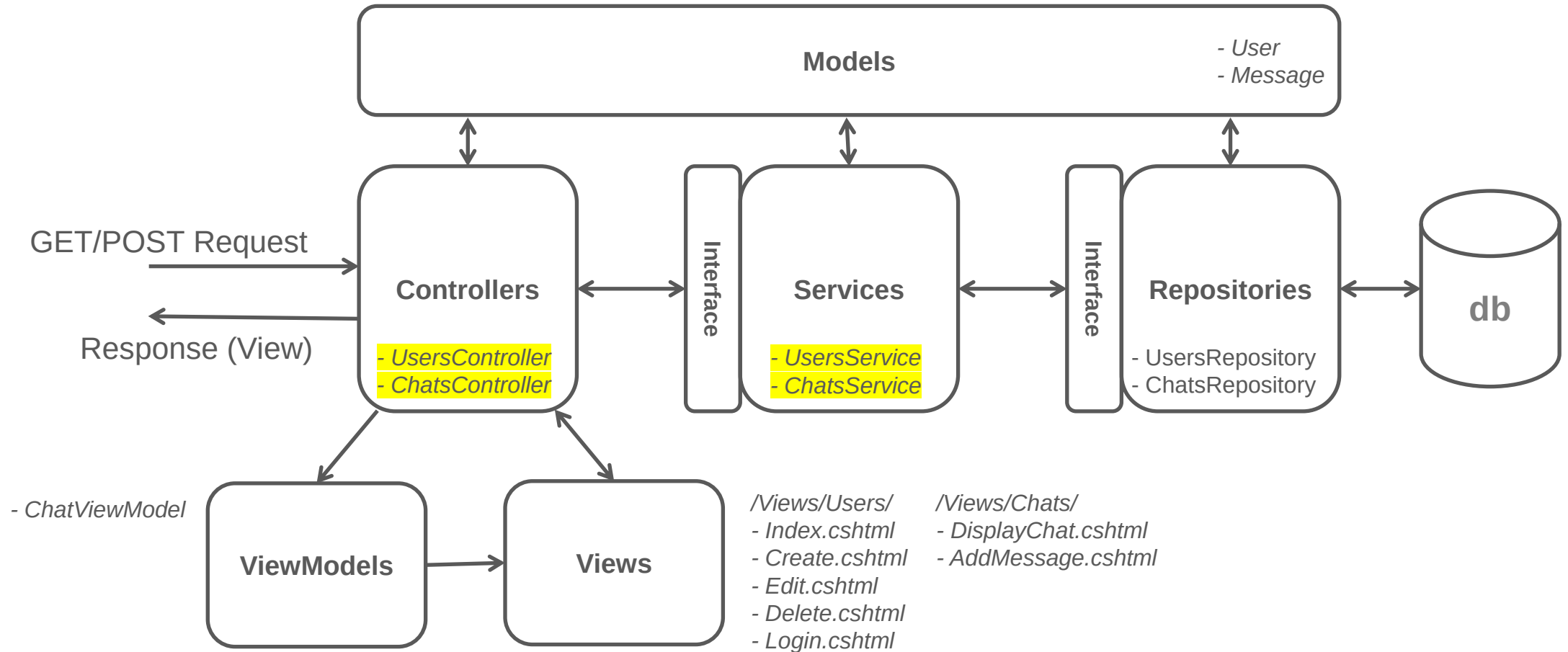
08 (wk-23) repetition

09 (wk-24) exams term 4

10 (wk-25) retake exams term 3

11 (wk-26) retake exams term 4

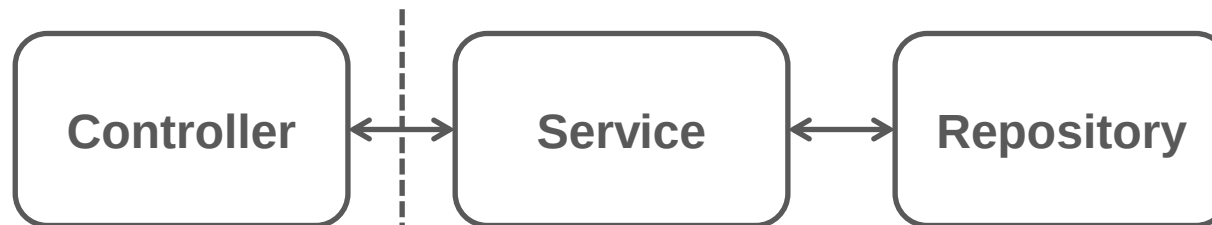
Architecture WhatsUp application



Adding a Service layer

Service layer

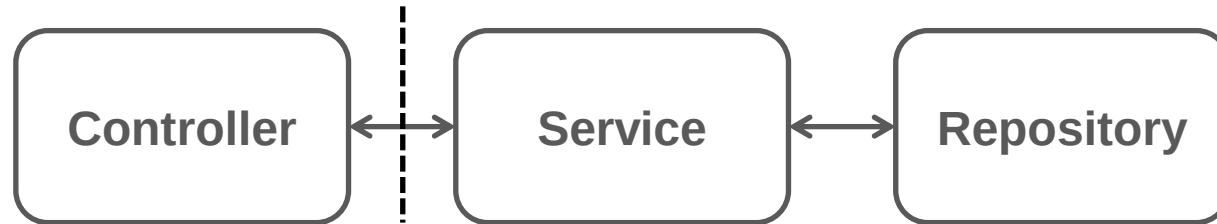
- Until now the WhatsUp architecture contains Controllers (handling the requests) and Repositories (containing the database code)
- A **Service layer** is now being added between the Controller and Repository
- This extra layer holds the core business logic (such as data processing and data validation), keeping the Controllers lightweight and focused on request handling



- The right-side part (Services, Repositories, database) can be seen as the backend (core) of the application, with methods like AddUser, SendMessage, GetChatMessages, ...
- The left-side part (Controllers and Views) can be seen as the frontend of the application, using the functionalities of the backend

Service layer

- Putting a Service between a Controller and a Repository means that the Controller communicates with the Service, and no longer with the Repository



- For each domain a separate Service will be created
- For the WhatsUp application that will be UsersService and ChatsService

Adding the UsersService

UserController → UserService → UserRepository

- The UsersController needs to have access to all functionalities we defined in the UserRepository
- Therefore, an **interface IUserService** is defined with the same methods as in IUserRepository

```
namespace MvcWhatsUp.Services
{
    public interface IUserService
    {
        List<User> GetAll();
        User? GetById(int userId);
        User? GetByLoginCredentials(string userName, string password);
        void Add(User user);
        void Update(User user);
        void Delete(User user);
    }
}
```

- The Controller will still be able to add, update, delete users, and to get all or specific users, but now through a service, not directly with the repository

UserService → UsersRepository

- And of course we need a **class UserService** to implement the interface IUserService
- The methods in the UserService are simply "pass-through" implementations (using the repository methods) →
- We will see more useful service implementations later
- As can be seen in the code, the constructor receives an object of type (interface) IUserRepository
- Which class is used for this object is already defined in Program.cs (Constructor Injection)

```
public class UserService : IUserService
{
    private IUserRepository _userRepository;

    public UserService(IUserRepository userRepository)
    {
        _userRepository = userRepository;
    }

    public List<User> GetAll()
    {
        return _userRepository.GetAll();
    }

    public User? GetById(int userId)
    {
        return _userRepository.GetById(userId);
    }

    public void Add(User user)
    {
        _userRepository.Add(user);
    }

    public void Update(User user)
    {
        _userRepository.Update(user);
    }

    public void Delete(User user)
    {
        _userRepository.Delete(user);
    }
}
```

UserController → UserService

- Once the service is available, the controller can use it
- Make sure that the UserController is now using namespace Service (instead of Repositories)!
- As can be seen in the code, the constructor receives an object of type (interface) IUserService

```
public class UserController : Controller
{
    private readonly IUserService _userService;

    public UserController(IUserService userService)
    {
        _userService = userService;
    }
}
```

- Which class is used for object creation must be defined in Program.cs (Constructor Injection)

```
// Add services to the container.
builder.Services.AddSingleton<IUsersRepository, DbUsersRepository>();
//builder.Services.AddSingleton<IUsersRepository, DummyUsersRepository>();
builder.Services.AddSingleton<IChatsRepository, DbChatsRepository>();

builder.Services.AddSingleton<IUserService, UserService>();
```

UserController → UsersService

- All repository method calls in the UsersController need be changed into service method calls
- Since interface IUserRepository is having the same methods as interface IUserService, this updating is an easy task

```
try
{
    // add user
    _userRepository.Add(user);

    // go back to user list (via Index)
    return RedirectToAction("Index");
}
```



```
try
{
    // add user
    _userService.Add(user);

    // go back to user list (via Index)
    return RedirectToAction("Index");
}
```

Class exercise

- Implement an interface `IUserService` and a class `UserService`
- Update the `UserController` in order to use the `UserService` instead of the `UsersRepository`
- Add a Constructor Injection definition in `Program.cs` for interface `IUserService`
- Test if your `WhatsUp` application still works

Adding the ChatsService

ChatsController → ChatsService → ChatsRepository

- The ChatsController needs to have access to all functionalities we defined in the ChatsRepository
- Therefore, an **interface IChatsService** is defined with the same methods as in IChatsRepository

```
namespace MvcWhatsUp.Services
{
    public interface IChatsService
    {
        void AddMessage(Message message);
        List<Message> GetMessages(int userId1, int userId2);
    }
}
```

- The Controller will still be able to add a message and to get all messages between 2 users, but now through a service, not directly with the repository

ChatsService → ChatsRepository

- And of course we need a **class ChatsService** to implement the interface IChatsService
- The methods in the ChatsService are simply "pass-through" implementations →
- We will see more useful service implementations later

```
public class ChatsService : IChatsService
{
    private readonly IChatsRepository _chatsRepository;

    public ChatsService(IChatsRepository chatsRepository)
    {
        _chatsRepository = chatsRepository;
    }

    public void AddMessage(Message message)
    {
        _chatsRepository.AddMessage(message);
    }

    public List<Message> GetMessages(int userId1, int userId2)
    {
        // get messages between 2 users
        return _chatsRepository.GetMessages(userId1, userId2);
    }
}
```

- As can be seen in the code, the constructor receives an object of type (interface) IChatsRepository
- Which class is used for this object is already defined in Program.cs (Constructor Injection)

ChatsController → ChatsService

- Once the service is available, the controller can use it
- Make sure that the ChatsController is now using namespace Service (instead of Repositories)!
- As can be seen in the code, the constructor receives an object of type (interface) IChatsService and IUsersService

```
public class ChatsController : Controller
{
    private readonly IChatsService _chatsService;
    private readonly IUsersService _usersService;

    public ChatsController(IChatsService chatsService, IUsersService usersService)
    {
        _chatsService = chatsService;
        _usersService = usersService;
    }
}
```

- Which class is used for object creation must be defined in Program.cs (Constructor Injection)

```
builder.Services.AddSingleton<IUsersService, UsersService>();
builder.Services.AddSingleton<IChatsService, ChatsService>();

builder.Services.AddControllersWithViews();
```

ChatsController → ChatsService

- All repository method calls in the ChatsController need be changed into service method calls
- Since the same methods are defined in interface IChatsService, this updating is an easy task

```
message.SendAt = DateTime.Now;  
_chatsRepository.AddMessage(message);
```



```
message.SendAt = DateTime.Now;  
_chatsService.AddMessage(message);
```

Class exercise

- Implement an interface IChatsService and a class ChatsService
- Update the ChatsController in order to use the ChatsService instead of the ChatsRepository
- Add a Constructor Injection definition in Program.cs for interface IChatsService

- Test if your WhatsUp application still works

Improving the UsersService

Improving the UserService

- Now that we have implemented a service layer (for domains Users and Chats), we can give it certain responsibilities, for example:
 - Input validation *(like checking if the email address of a new user is valid)*
 - Business rule validation *(like checking if the email address of a new user is unique)*
 - Logging *(it is good practice to keep track of everything that was done in an application)*
 - Caching *(in order to lower the amount of database communication/traffic)*
 - Security *(like hashing passwords in the database)*
- We will do some **business rule validation** and **password hashing** in the UserService

Business rule validation in the UsersService

Business rule validation in the UsersService

- As a (business) rule in the WhatsUp application, it should not be possible to have multiple users with the same email address (so, the email address should be unique)
- When a new user is about to be added to the application, there should be a check if there is already a user with the same email address
- A good place to implement this check is the service layer
- The UsersService should be able to ask the repository if a certain email address already exists, so we need to add this to interface IUsersRepository

```
public interface IUsersRepository
{
    List<User> GetAll();
    User? GetById(int userId);
    User? GetByLoginCredentials(string username, string password);
    void Add(User user);
    void Update(User user);
    void Delete(User user);

    bool EmailAddressExists(string emailAddress);
}
```

Business rule validation in the UserService

- For implementing the EmailAddressExists in class DbUsersRepository, you can use an SQL query like: "SELECT COUNT(*) FROM Users WHERE EmailAddress = @EmailAddress", and get the count via command.ExecuteScalar()
- In the UserService, a test on email address existence should of course be done before adding the user

```
public void Add(User user)
{
    // make sure that email address is unique
    if (_usersRepository.EmailAddressExists(user.EmailAddress))
        throw new Exception("Email address is already in use!");
    _usersRepository.Add(user);
}
```

- What other method should be changed in order to make sure the email address stays unique!?!?

Password hashing in the UsersService

Password hashing in the UserService

- As mentioned before (Web Programming 1) a password should never be stored in clear text in the database → if someone gains access to the database, you will have a problem...
- Hashing is typically used for handling passwords
- Hashing is a one-way process, meaning you can't decrypt the password back to its original form
- Instead, when a user logs in, you hash the entered password and compare it to the stored hash
- Here's a simple implementation of a Hash method (in the UserService):

```
private string HashPassword(string password)
{
    using (SHA256 sha256 = SHA256.Create())
    {
        byte[] hashBytes = sha256.ComputeHash(Encoding.UTF8.GetBytes(password));
        return Convert.ToBase64String(hashBytes);
    }
}
```

Password hashing in the UsersService

- There are 2 methods in the UsersService that needs to be changed:
 1. **Add** (*password should be hashed before storing it in the database*)
 2. **GetUserByLoginCredentials** (*the entered password needs to be hashed before retrieving the user from the database*)
- In the **Add** method of the UsersService the password is hashed before calling the repository
- Since we don't want to change the original user object, we are using a copy (created by using a "copy constructor")

```
public void Add(User user)
{
    // make sure that email address is unique
    if (_usersRepository.EmailAddressExists(user.EmailAddress))
        throw new Exception("Email address is already in use!");

    // make sure the hashed password is stored
    User copyUser = new User(user);
    copyUser.Password = HashPassword(user.Password);

    _usersRepository.Add(copyUser);

    // make sure that if id is auto generated, it is set in the user
    if (user.UserId != copyUser.UserId)
        user.UserId = copyUser.UserId;
}
```

Password hashing in the UsersService

- Below you can see some passwords in clear text... 🤔 and a hashed password

	UserId	UserName	MobileNumber	EmailAddress	Password
1	1	Thom Yorke	06-29187211	thom.yorke@gmail.com	test123
2	2	Jonny Greenwood	06-92887214	jonny.greenwood@gmail.com	test123
3	3	Colin Greenwood	06-98746320	colin.greenwood@gmail.com	test123
4	4	Phil Selway	06-25847362	phil.selway@gmail.com	test123
5	5	Ed O'Brien	06-19926651	ed.obrien@gmail.com	test123
6	9	Gerwin van Dijken	06-29187211	gerwin.vandijken@inholland.nl	7NcYcNGWMxapfjrDQlyYNa2M8PPBvHA1J8MCZVNPda4=



- When changing an application to use hashed passwords, existing passwords in clear text needs to be updated in the database!

Password hashing in the UsersService

- The other method to update in the UsersService is **GetByLoginCredentials**
- This method receives the username and password as parameters
- Since the passwords are now hashed in the database, all we need to do here is to pass the hashed password to the repository

```
public User? GetByLoginCredentials(string userName, string password)
{
    return _usersRepository.GetByLoginCredentials(userName, HashPassword(password));
}
```

- Make sure to understand that first the clear passwords were compared with each other (in the database), now the hashed passwords are compared
- The repository method GetByLoginCredentials does not have to change

Homework – week 3

- Implement an interface `IUserService` and a class `UserService`
- Update the `UserController` in order to use the `UserService` instead of the `UsersRepository`
- Add a Constructor Injection definition in `Program.cs` for interface `IUserService`

- Implement an interface `IChatsService` and a class `ChatsService`
- Update the `ChatsController` in order to use the `ChatsService` instead of the `ChatsRepository`
- Add a Constructor Injection definition in `Program.cs` for interface `IChatsService`

- Add business rule validation to the `UserService` in order to keep email addresses unique
- Add passwords hashing to the `UserService` in order to have unreadable passwords in the db

